

PROJET 3IIR

Conception et réalisation d'un système de suivi des patients pour un cabinet dentaire

Ingénierie Informatique et Réseaux

Réalisé par :

-Laknifli Khalil

-Ait Elmoumen Aymane

-Asli Oubaydoullah

-El Fellah Adil

-Ourhanim Ibrahim

Encadré par :

-Pr.Safsouf Yassine

-Pr.Hazman Chaimae

-Pr.Ait Rai Khaoula

-Pr.El Gourari Abelali

Résumé

Résumé :

Ce rapport présente l'analyse et la conception complète d'un système de gestion de cabinet dentaire. L'étude couvre l'analyse des besoins fonctionnels et non fonctionnels, la modélisation UML du système à travers des diagrammes de cas d'utilisation, de séquence et de classes, ainsi que la conception d'une base de données relationnelle normalisée. Le système permet de gérer les patients, les rendez-vous, les traitements, le personnel et les notifications pour différents types d'utilisateurs : patients, docteurs et personnel administratif. Les résultats montrent une architecture cohérente et extensible, avec une base de données optimisée et des modèles UML complets qui constituent une base solide pour l'implémentation future du système.

Mots clés : Système de gestion, Cabinet dentaire, UML, Base de données, Modélisation, Conception

Abstract

Abstract :

This report presents the complete analysis and design of a dental clinic management system. The study covers the analysis of functional and non-functional requirements, UML modeling of the system through use case, sequence and class diagrams, as well as the design of a normalized relational database. The system allows managing patients, appointments, treatments, staff and notifications for different types of users : patients, doctors and administrative staff. The results show a coherent and extensible architecture, with an optimized database and complete UML models that constitute a solid foundation for the future implementation of the system.

Keywords : Management system, Dental clinic, UML, Database, Modeling, Design

Table des matières

Résumé	i
Abstract	ii
Liste des Abréviations	v
1 Introduction Générale	1
I. Contexte du Projet	1
II. Objectifs du Projet	1
III. Structure du Document	1
2 Analyse des Besoins	3
I. Analyse Fonctionnelle	3
I.1. Acteurs du Système	3
I.2. Fonctionnalités Principales	4
II. Analyse Non Fonctionnelle	5
II.1. Exigences de Performance	5
II.2. Exigences de Sécurité	5
II.3. Exigences de Disponibilité	5
III. Contraintes	6
3 Conception UML	7
I. Diagrammes de Conception	7
I.1. Diagramme de Cas d'Utilisation	7
I.2. Diagramme de Séquence	10

I.3.	Diagramme de Classes UML	12
II.	Correspondance avec la Base de Données	14
4	Modèle de Données et Base de Données	16
I.	Schéma de Base de Données.....	16
I.1.	Architecture de la Base de Données.....	16
I.2.	Tables Principales.....	16
I.3.	Relations et Contraintes.....	22
I.4.	Vues	23
I.5.	Optimisations	24
I.6.	Données d'Exemple	24
5	Conclusion Générale	26
I.	Résumé	26
II.	Points Forts de la Conception	26
III.	Correspondance entre les Modèles.....	27
IV.	Perspectives	27

Table des figures

UML Unified Modeling Language

PFA Projet de Fin d'Année

API Application Programming Interface

SQL Structured Query Language

3NF Troisième Forme Normale

Table des figures

3.1	Diagramme de cas d'utilisation du système de gestion de cabinet dentaire . . .	8
3.2	Diagramme de séquence : Processus de gestion des rendez-vous et traitements .	10
3.3	Diagramme de classes du système de gestion de cabinet dentaire	12

Chapitre 1

Introduction Générale

I. Contexte du Projet

Ce projet consiste en l'analyse et la conception d'un système complet de gestion de cabinet dentaire. Le système doit permettre de gérer les patients, les rendez-vous, les traitements, le personnel et les notifications pour différents types d'utilisateurs : patients, docteurs et personnel administratif.

Ce document présente l'analyse des besoins, la modélisation UML du système et la conception de la base de données relationnelle.

II. Objectifs du Projet

Les objectifs principaux de ce projet sont :

- Analyser les besoins fonctionnels et non fonctionnels du système
- Identifier les acteurs et leurs interactions avec le système
- Modéliser le système à l'aide de diagrammes UML (cas d'utilisation, séquence, classes)
- Concevoir une base de données relationnelle normalisée et optimisée
- Définir les relations entre les entités du système

III. Structure du Document

Ce rapport est organisé comme suit :

- **Chapitre 2** : Analyse des besoins et des acteurs

- **Chapitre 3** : Conception UML (diagrammes de cas d'utilisation, séquence et classes)
- **Chapitre 4** : Modèle de données et base de données
- **Chapitre 5** : Conclusion

Chapitre 2

Analyse des Besoins

I. Analyse Fonctionnelle

I.1. Acteurs du Système

Le système identifie quatre acteurs principaux :

I.1.1. Patient

Le patient est l'utilisateur principal qui bénéficie des services du cabinet dentaire. Il peut :

- Créer un compte et s'authentifier
- Consulter son dossier médical
- Prendre, modifier et annuler des rendez-vous
- Modifier ses informations personnelles
- Recevoir des notifications et rappels

I.1.2. Docteur

Le docteur est le professionnel médical qui fournit les soins. Il peut :

- Consulter le tableau de bord avec les statistiques
- Voir tous les rendez-vous des patients
- Consulter les dossiers médicaux des patients
- Ajouter et modifier des traitements
- Consulter le planning des rendez-vous
- Recevoir des notifications

I.1.3. Personnel Administratif

Le personnel administratif gère les aspects administratifs du cabinet. Il peut :

- Gérer les patients (création, modification, consultation)
- Gérer les rendez-vous (planification, modification, annulation)
- Recevoir des notifications sur les actions des patients
- Gérer les rappels pour les patients

I.1.4. Système de Notifications

Le système de notifications est un acteur externe qui gère l'envoi automatique de :

- Confirmations de rendez-vous
- Rappels automatiques (24h avant le rendez-vous)
- Notifications au personnel administratif

I.2. Fonctionnalités Principales

I.2.1. Gestion des Comptes et Authentification

- Création de compte pour les patients
- Authentification sécurisée pour tous les utilisateurs
- Gestion des rôles et permissions
- Déconnexion

I.2.2. Gestion des Rendez-vous

- Prise de rendez-vous en ligne par les patients
- Vérification de la disponibilité des créneaux
- Modification de rendez-vous existants
- Annulation de rendez-vous
- Consultation du planning par les docteurs
- Gestion des rendez-vous par le personnel administratif

I.2.3. Gestion des Dossiers Médicaux

- Consultation du dossier médical par le patient
- Consultation et modification du dossier par le docteur

- Gestion des antécédents médicaux
- Gestion des allergies
- Historique des traitements

I.2.4. Gestion des Traitements

- Ajout de traitements par le docteur
- Modification des traitements
- Association des traitements aux dossiers médicaux
- Gestion des prescriptions
- Génération de factures

I.2.5. Système de Notifications

- Envoi automatique de confirmations de rendez-vous
- Rappels automatiques 24h avant les rendez-vous
- Notifications au personnel lors d'actions importantes
- Notifications aux patients pour les rappels

II. Analyse Non Fonctionnelle

II.1. Exigences de Performance

- Temps de réponse acceptable pour les opérations courantes (< 2 secondes)
- Support de plusieurs utilisateurs simultanés
- Gestion efficace des requêtes de base de données

II.2. Exigences de Sécurité

- Authentification sécurisée des utilisateurs
- Gestion des rôles et permissions
- Protection des données médicales sensibles
- Chiffrement des mots de passe

II.3. Exigences de Disponibilité

- Système disponible pendant les heures d'ouverture du cabinet

- Sauvegarde régulière des données
- Récupération en cas de panne

III. Contraintes

- Respect de la confidentialité des données médicales
- Conformité aux réglementations sur la protection des données
- Interface intuitive pour tous les types d'utilisateurs
- Compatibilité avec les navigateurs web modernes

Chapitre 3

Conception UML

I. Diagrammes de Conception

Le projet inclut trois diagrammes UML essentiels qui modélisent l'architecture et le comportement du système. Ces diagrammes sont essentiels pour comprendre la structure du système et les interactions entre ses différents composants.

I.1. Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation présente les fonctionnalités du système du point de vue des différents acteurs. Il identifie les interactions entre les utilisateurs (acteurs) et le système.

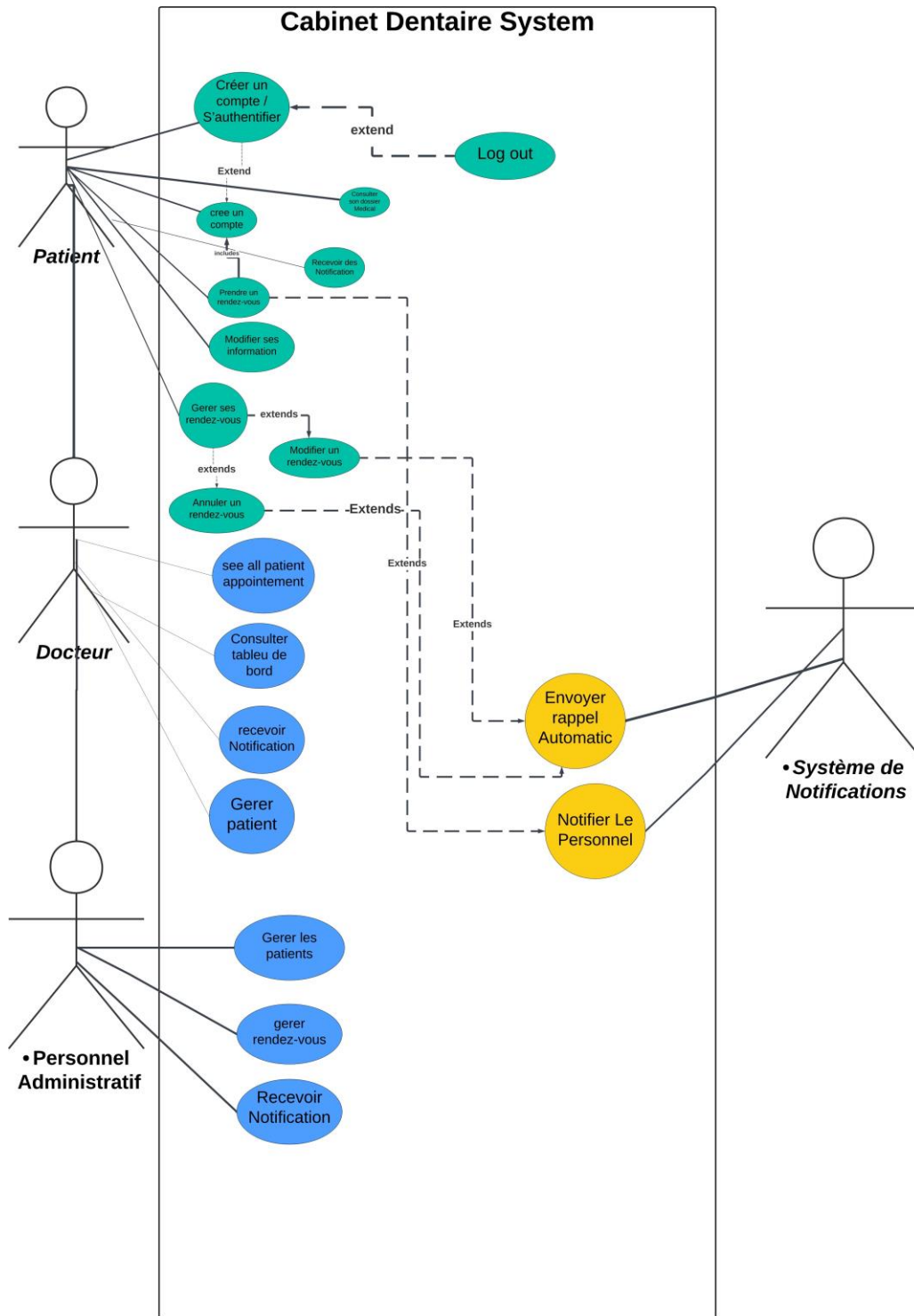


Figure 3.1. Diagramme de cas d'utilisation du système de gestion de cabinet dentaire

I.1.1. Acteurs Identifiés

Le diagramme illustre les acteurs suivants :

- **Patient** : Utilisateur principal qui peut créer un compte, prendre des rendez-vous, consulter son dossier médical, et recevoir des notifications
- **Docteur** : Professionnel médical qui peut consulter les dossiers patients, gérer les traitements, et consulter le tableau de bord
- **Personnel Administratif** : Personnel qui gère les patients et les rendez-vous, et reçoit des notifications
- **Système de Notifications** : Système externe responsable de l'envoi des notifications

I.1.2. Cas d'Utilisation Principaux

Les cas d'utilisation principaux incluent :

Pour le Patient :

- Créer un compte / S'authentifier
- Consulter son dossier médical
- Prendre un rendez-vous
- Modifier ses informations
- Gérer ses rendez-vous (modifier, annuler)
- Recevoir des notifications
- Se déconnecter

Pour le Docteur :

- Voir tous les rendez-vous des patients
- Consulter le tableau de bord
- Recevoir des notifications
- Gérer les patients

Pour le Personnel Administratif :

- Gérer les patients
- Gérer les rendez-vous
- Recevoir des notifications

Cas d'Utilisation Système :

- Envoyer des rappels automatiques (déclenché lors de la création, modification ou annulation de rendez-vous)
- Notifier le personnel (déclenché lors de l'envoi de rappels automatiques)

I.1.3. Relations entre Cas d'Utilisation

Les relations *includes* et *extends* montrent les dépendances entre les cas d'utilisation :

- Créer un compte / S'authentifier includes Créer un compte
- Gérer ses rendez-vous extends Modifier un rendez-vous
- Gérer ses rendez-vous extends Annuler un rendez-vous
- Envoyer rappel automatique extends Prendre un rendez-vous
- Envoyer rappel automatique extends Modifier un rendez-vous
- Envoyer rappel automatique extends Annuler un rendez-vous
- Notifier le Personnel extends Envoyer rappel automatique

I.2. Diagramme de Séquence

Le diagramme de séquence illustre les interactions temporelles entre les différents objets du système lors de l'exécution d'un scénario.

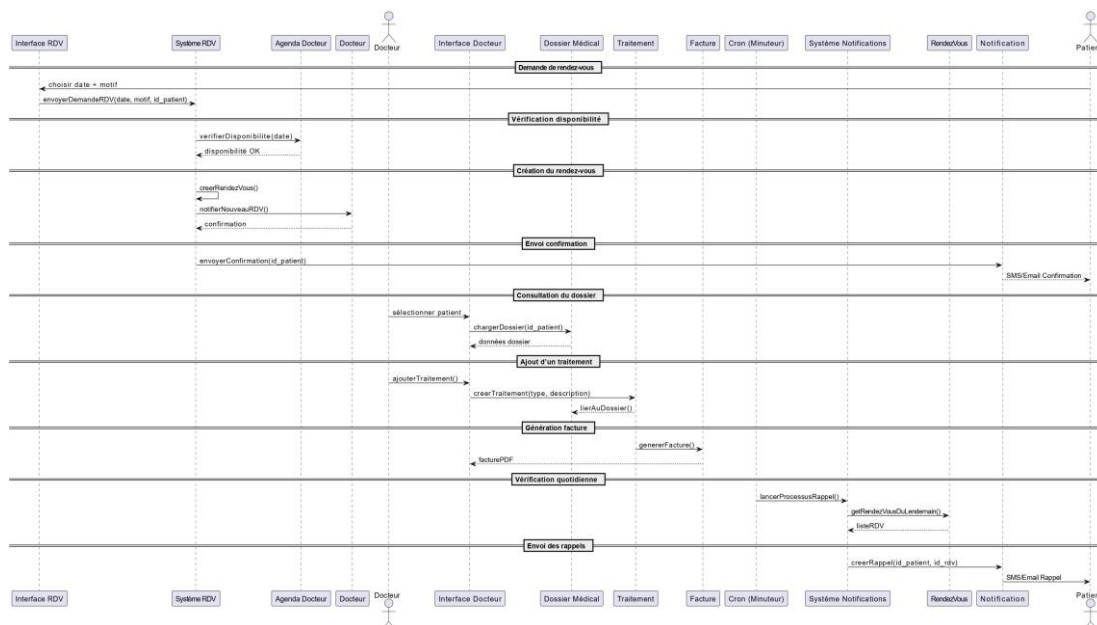


Figure 3.2. Diagramme de séquence : Processus de gestion des rendez-vous et traitements

I.2.1. Scénarios Modélisés

Ce diagramme montre le flux complet d'un processus de rendez-vous, incluant :

1) Demande de rendez-vous :

- L'interface permet au patient de choisir une date et un motif
- Le système vérifie la disponibilité dans l'agenda du docteur

2) Création du rendez-vous :

- Une fois la disponibilité confirmée, le rendez-vous est créé

- Le docteur est notifié de la création du nouveau rendez-vous

3) **Envoi de confirmation :**

- Le système de notifications envoie une confirmation au patient par SMS ou email

4) **Consultation du dossier :**

- Le docteur peut consulter le dossier médical d'un patient via son interface
- Le dossier médical charge les données du patient

5) **Ajout de traitement :**

- Le docteur peut ajouter un traitement
- Le traitement est créé avec type et description
- Le traitement est lié au dossier médical du patient

6) **Génération de facture :**

- Un traitement peut déclencher la génération automatique d'une facture PDF

7) **Rappels automatiques :**

- Un processus cron (minuteur) vérifie quotidiennement les rendez-vous du lendemain
- Des rappels automatiques sont envoyés aux patients par SMS ou email

I.2.2. Participants du Diagramme

Les participants principaux dans ce diagramme sont :

- **Interface RDV** : Interface utilisateur pour la gestion des rendez-vous
- **Système RDV** : Système de gestion des rendez-vous (logique métier)
- **Agenda Docteur** : Gestion de l'agenda et de la disponibilité du docteur
- **Docteur** : Acteur externe (professionnel médical)
- **Interface Docteur** : Interface utilisateur pour le docteur
- **Dossier Médical** : Gestion des dossiers médicaux des patients
- **Traitement** : Gestion des traitements médicaux
- **Facture** : Génération des factures
- **Cron (Minuteur)** : Processus automatisé pour les tâches planifiées
- **Système Notifications** : Gestion de l'envoi des notifications
- **RendezVous** : Entité/module représentant les rendez-vous
- **Notification** : Entité/module représentant les notifications
- **Patient** : Acteur externe recevant les confirmations et rappels

I.3. Diagramme de Classes UML

Le diagramme de classes représente la structure statique du système en montrant les classes, leurs attributs, leurs méthodes et les relations entre elles.

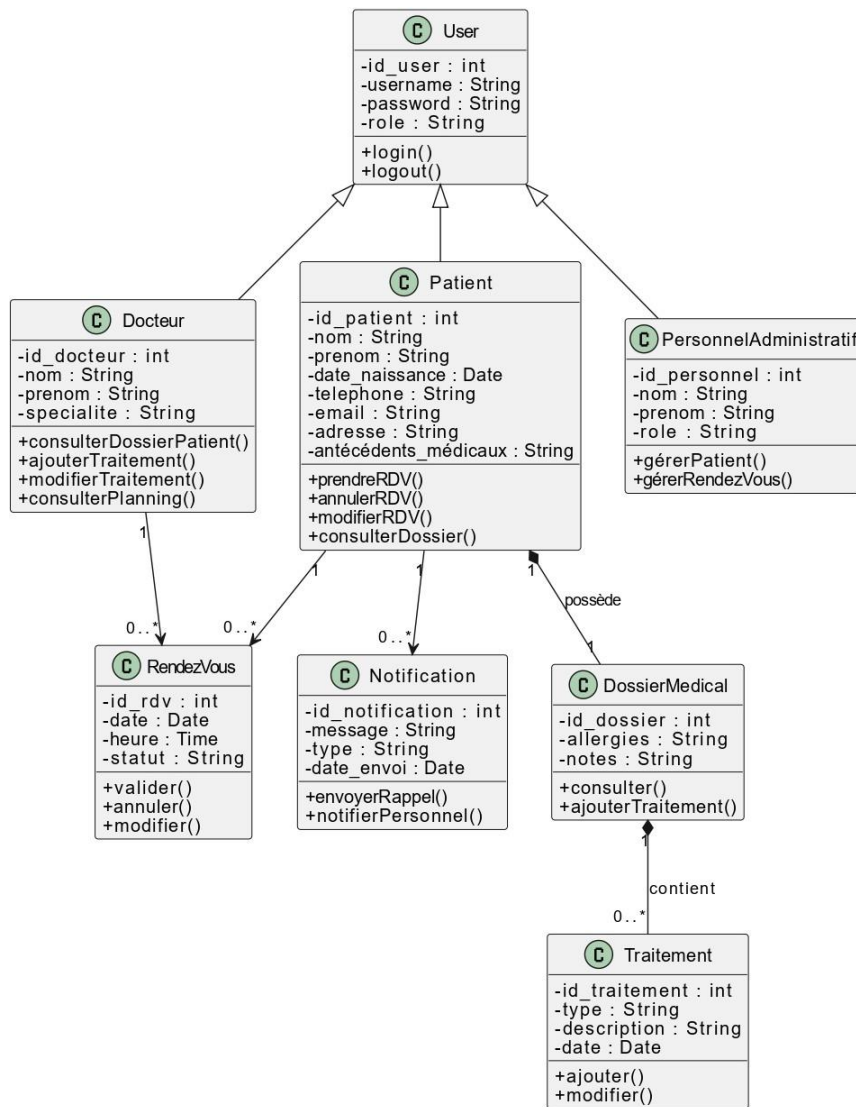


Figure 3.3. Diagramme de classes du système de gestion de cabinet dentaire

I.3.1. Hiérarchie d'Héritage

Classe de Base : User La classe `User` est la classe parente qui définit les attributs et méthodes communs à tous les utilisateurs :

Attributs :

- `id_user` : Identifiant unique (int)
- `username` : Nom d'utilisateur (String)
- `password` : Mot de passe (String)
- `role` : Rôle de l'utilisateur (String)

Méthodes :

- `login()` : Authentification de l'utilisateur
- `logout()` : Déconnexion de l'utilisateur

Classes Dérivées

- **Docteur** : Hérite de `User` et ajoute :
 - Attributs : `id_docteur`, `nom`, `prenom`, `specialite`
 - Méthodes : `consulterDossierPatient()`, `ajouterTraitement()`, `modifierTrai`
`consulterPlanning()`
- **Patient** : Hérite de `User` et ajoute :
 - Attributs : `id_patient`, `nom`, `prenom`, `date_naissance`, `telephone`, `email`,
`adresse`, `antécédents_médicaux`
 - Méthodes : `prendreRDV()`, `annulerRDV()`, `modifierRDV()`, `consulterDossier()`
- **PersonnelAdministratif** : Hérite de `User` et ajoute :
 - Attributs : `id_personnel`, `nom`, `prenom`, `role`
 - Méthodes : `gérerPatient()`, `gérerRendezVous()`

I.3.2. Classes Métier

RendezVous Gère les rendez-vous avec :

- Attributs : `id_rdv`, `date`, `heure`, `statut`
- Méthodes : `valider()`, `annuler()`, `modifier()`

Notification Gère l'envoi de rappels et les notifications au personnel :

- Attributs : `id_notification`, `message`, `type`, `date_envoi`
- Méthodes : `envoyerRappel()`, `notifierPersonnel()`

DossierMedical Contient les informations médicales et les traitements :

- Attributs : `id_dossier`, `allergies`, `notes`
- Méthodes : `consulter()`, `ajouterTraitement()`

Traitement Représente un traitement médical :

- Attributs : `id_traitement`, `type`, `description`, `date`
- Méthodes : `ajouter()`, `modifier()`

I.3.3. Relations entre Classes

Relations d'Association

- **Docteur - RendezVous :**
 - Un Docteur peut avoir plusieurs RendezVous (0..*)
 - Un RendezVous est associé à exactement un Docteur (1)
- **Patient - RendezVous :**
 - Un Patient peut avoir plusieurs RendezVous (0..*)
 - Un RendezVous est associé à exactement un Patient (1)
- **Patient - Notification :**
 - Un Patient peut recevoir plusieurs Notifications (0..*)
 - Une Notification est destinée à exactement un Patient (1)
- **Patient - DossierMedical :**
 - Relation de possession (1-1) : chaque Patient possède exactement un DossierMedical
 - Un DossierMedical appartient à exactement un Patient (1)
- **DossierMedical - Traitement :**
 - Un DossierMedical contient plusieurs Traitements (0..*)
 - Un Traitement appartient à exactement un DossierMedical (1)

II. Correspondance avec la Base de Données

Le diagramme de classes constitue la base du modèle de données implémenté dans la base de données MySQL. Il existe des correspondances directes entre les classes et les tables de la base de données :

- User → Table `users`
- Docteur → Table `staff` (avec `role='dentist'`)

- Patient → **Table** patients
- PersonnelAdministratif → **Table** staff (avec role='receptionist' ou 'admin')
- RendezVous → **Table** appointments
- Notification → **Tables** notifications et admin_notifications
- DossierMedical → **Informations dans la table** patients et **table** patient_allergies
- Traitement → **Table** treatments

Chapitre 4

Modèle de Données et Base de Données

I. Schéma de Base de Données

Le système utilise une base de données MySQL relationnelle avec un schéma normalisé. La base de données `cabinet_dentaire` contient 11 tables principales et 3 vues pour faciliter les requêtes.

I.1. Architecture de la Base de Données

La base de données suit les principes de normalisation (3NF) pour éviter la redondance des données et assurer l'intégrité référentielle. Le schéma utilise le charset UTF8MB4 pour supporter tous les caractères Unicode, notamment les caractères arabes et français.

I.2. Tables Principales

I.2.1. Table `users`

Cette table stocke tous les utilisateurs du système (patients, docteurs, personnel administratif).

Structure :

- `id` : Identifiant unique (VARCHAR(50), PRIMARY KEY)
- `email` : Email unique de l'utilisateur (VARCHAR(255), UNIQUE, NOT NULL)
- `password` : Mot de passe (VARCHAR(255), NOT NULL) - À hasher en production
- `role` : Rôle de l'utilisateur (ENUM : 'patient', 'docteur', 'personnelAdministratif', NOT NULL)
- `first_name`, `last_name` : Nom et prénom (VARCHAR(100), NOT NULL)

- `patient_id` : Référence vers la table `patients` (VARCHAR(50), NULL, FOREIGN KEY)
- `staff_id` : Référence vers la table `staff` (VARCHAR(50), NULL, FOREIGN KEY)
- `created_at`, `updated_at` : Timestamps automatiques

Index :

- Index sur `email` pour les recherches rapides
- Index sur `role` pour filtrer par type d'utilisateur
- Index sur `patient_id` et `staff_id` pour les jointures

I.2.2. Table `patients`

Stocke les informations complètes des patients.

Structure :

- `id` : Identifiant unique (VARCHAR(50), PRIMARY KEY)
- `first_name`, `last_name` : Nom et prénom (VARCHAR(100), NOT NULL)
- `date_of_birth` : Date de naissance (DATE, NOT NULL)
- `phone` : Téléphone (VARCHAR(20), NOT NULL)
- `email` : Email unique (VARCHAR(255), UNIQUE, NOT NULL)
- `address` : Adresse complète (TEXT, NOT NULL)
- `blood_type` : Groupe sanguin (VARCHAR(10), NULL)
- `medical_history` : Antécédents médicaux (TEXT, NULL)
- `registration_date` : Date d'inscription (DATE, NOT NULL)
- `created_at`, `updated_at` : Timestamps automatiques

Index :

- Index sur `email` et `phone` pour les recherches
- Index sur `registration_date` pour les statistiques
- Index composite sur `last_name`, `first_name` pour les recherches par nom

I.2.3. Table `patient_allergies`

Table de liaison pour gérer les allergies multiples d'un patient (relation 1-N).

Structure :

- `id` : Identifiant unique (INT, AUTO_INCREMENT, PRIMARY KEY)
- `patient_id` : Référence vers `patients` (VARCHAR(50), NOT NULL, FOREIGN KEY)
- `allergy` : Nom de l'allergie (VARCHAR(255), NOT NULL)

- `created_at` : Timestamp automatique

Contrainte : ON DELETE CASCADE pour supprimer les allergies lors de la suppression d'un patient.

I.2.4. Table **staff**

Gère le personnel du cabinet (dentistes, assistants, réceptionnistes, administrateurs).

Structure :

- `id` : Identifiant unique (VARCHAR(50), PRIMARY KEY)
- `first_name`, `last_name` : Nom et prénom (VARCHAR(100), NOT NULL)
- `role` : Type de personnel (ENUM : 'dentist', 'assistant', 'receptionist', 'admin', NOT NULL)
- `specialty` : Spécialité (VARCHAR(100), NULL) - Principalement pour les dentistes
- `phone` : Téléphone (VARCHAR(20), NOT NULL)
- `email` : Email unique (VARCHAR(255), UNIQUE, NOT NULL)
- `created_at`, `updated_at` : Timestamps automatiques

Index :

- Index sur `email` pour les recherches
- Index sur `role` pour filtrer par type de personnel
- Index composite sur `last_name`, `first_name` pour les recherches par nom

I.2.5. Table **staff_schedules**

Gère les horaires de travail du personnel (relation 1-N).

Structure :

- `id` : Identifiant unique (INT, AUTO_INCREMENT, PRIMARY KEY)
- `staff_id` : Référence vers `staff` (VARCHAR(50), NOT NULL, FOREIGN KEY)
- `day_of_week` : Jour de la semaine (VARCHAR(20), NOT NULL)
- `created_at` : Timestamp automatique

Contrainte : UNIQUE KEY sur (`staff_id`, `day_of_week`) pour éviter les doublons.

I.2.6. Table **appointments**

Gère tous les rendez-vous du cabinet.

Structure :

- `id` : Identifiant unique (VARCHAR(50), PRIMARY KEY)
- `patient_id` : Référence vers le patient (VARCHAR(50), NOT NULL, FOREIGN KEY)
- `dentist_id` : Référence vers le dentiste (VARCHAR(50), NOT NULL, FOREIGN KEY)
- `appointment_date` : Date du rendez-vous (DATE, NOT NULL)
- `appointment_time` : Heure du rendez-vous (TIME, NOT NULL)
- `duration` : Durée en minutes (INT, NOT NULL)
- `type` : Type de rendez-vous (VARCHAR(100), NOT NULL) - Ex : Consultation, Détartrage, Extraction
- `status` : Statut (ENUM : 'scheduled', 'completed', 'cancelled', 'no-show', NOT NULL, DEFAULT 'scheduled')
- `notes` : Notes optionnelles (TEXT, NULL)
- `created_at`, `updated_at` : Timestamps automatiques

Contraintes :

- FOREIGN KEY sur `patient_id` avec ON DELETE CASCADE
- FOREIGN KEY sur `dentist_id` avec ON DELETE RESTRICT (empêche la suppression d'un dentiste avec des rendez-vous)

Index :

- Index sur `patient_id` et `dentist_id` pour les jointures
- Index sur `appointment_date` pour les requêtes par date
- Index sur `status` pour filtrer par statut
- Index composite sur (`appointment_date`, `appointment_time`) pour vérifier les conflits

I.2.7. Table `treatments`

Stocke tous les traitements effectués.

Structure :

- `id` : Identifiant unique (VARCHAR(50), PRIMARY KEY)
- `patient_id` : Référence vers le patient (VARCHAR(50), NOT NULL, FOREIGN KEY)
- `dentist_id` : Référence vers le dentiste (VARCHAR(50), NOT NULL, FOREIGN KEY)
- `treatment_date` : Date du traitement (DATE, NOT NULL)

- `type` : Type de traitement (VARCHAR(100), NOT NULL) - Ex : Plombage, Détartrage, Couronne
- `tooth` : Numéro ou position de la dent (VARCHAR(20), NULL)
- `description` : Description détaillée (TEXT, NOT NULL)
- `cost` : Coût du traitement (DECIMAL(10,2), NOT NULL)
- `next_visit` : Date de la prochaine visite recommandée (DATE, NULL)
- `created_at`, `updated_at` : Timestamps automatiques

Index :

- Index sur `patient_id` et `dentist_id` pour les jointures
- Index sur `treatment_date` pour les requêtes par date
- Index sur `type` pour les statistiques par type de traitement

I.2.8. Table `treatment_prescriptions`

Gère les prescriptions associées aux traitements (relation 1-N).

Structure :

- `id` : Identifiant unique (INT, AUTO_INCREMENT, PRIMARY KEY)
- `treatment_id` : Référence vers `treatments` (VARCHAR(50), NOT NULL, FOREIGN KEY)
- `prescription` : Texte de la prescription (TEXT, NOT NULL)
- `created_at` : Timestamp automatique

Contrainte : ON DELETE CASCADE pour supprimer les prescriptions lors de la suppression d'un traitement.

I.2.9. Table `reminders`

Gère les rappels pour les patients.

Structure :

- `id` : Identifiant unique (VARCHAR(50), PRIMARY KEY)
- `patient_id` : Référence vers le patient (VARCHAR(50), NOT NULL, FOREIGN KEY)
- `type` : Type de rappel (ENUM : 'checkup', 'treatment', 'followup', NOT NULL)
- `due_date` : Date d'échéance (DATE, NOT NULL)
- `message` : Message du rappel (TEXT, NOT NULL)
- `status` : Statut (ENUM : 'pending', 'sent', 'completed', NOT NULL, DEFAULT 'pending')

- `created_at`, `updated_at` : Timestamps automatiques

Index :

- Index sur `patient_id` pour les jointures
- Index sur `due_date` pour les requêtes de rappels à venir
- Index sur `status` pour filtrer par statut

I.2.10. Table `notifications`

Notifications pour les utilisateurs (patients, docteurs, personnel).

Structure :

- `id` : Identifiant unique (VARCHAR(50), PRIMARY KEY)
- `user_id` : Référence vers l'utilisateur (VARCHAR(50), NOT NULL, FOREIGN KEY)
- `type` : Type de notification (ENUM : 'appointment', 'reminder', 'system', 'appointment_cancelled', 'appointment_modified', 'appointment_created', NOT NULL)
- `title` : Titre de la notification (VARCHAR(255), NOT NULL)
- `message` : Message de la notification (TEXT, NOT NULL)
- `read` : Statut de lecture (BOOLEAN, DEFAULT FALSE)
- `related_id` : ID de l'entité liée (VARCHAR(50), NULL) - Ex : ID du rendez-vous
- `created_at` : Timestamp automatique

Index :

- Index sur `user_id` pour les requêtes par utilisateur
- Index sur `read` pour filtrer les notifications non lues
- Index sur `created_at` pour le tri chronologique
- Index sur `type` pour filtrer par type

I.2.11. Table `admin_notifications`

Notifications administratives pour le personnel administratif.

Structure :

- `id` : Identifiant unique (VARCHAR(50), PRIMARY KEY)
- `type` : Type de notification (ENUM : 'appointment_created', 'appointment_cancelled', 'appointment_modified', 'patient_registered', NOT NULL)
- `title` : Titre de la notification (VARCHAR(255), NOT NULL)
- `message` : Message de la notification (TEXT, NOT NULL)
- `read` : Statut de lecture (BOOLEAN, DEFAULT FALSE)

- `related_id` : ID de l'entité liée (VARCHAR(50), NULL)
- `patient_id` : Référence optionnelle vers le patient (VARCHAR(50), NULL, FOREIGN KEY)
- `created_at` : Timestamp automatique

Index :

- Index sur `read` pour filtrer les notifications non lues
- Index sur `created_at` pour le tri chronologique
- Index sur `type` pour filtrer par type
- Index sur `patient_id` pour les jointures

I.3. Relations et Contraintes

I.3.1. Clés Étrangères

Les relations entre tables sont gérées par des clés étrangères avec les actions suivantes :

- **ON DELETE CASCADE** : Pour les relations patients (suppression en cascade des données liées)
 - `patient_allergies` → `patients`
 - `appointments` → `patients`
 - `treatments` → `patients`
 - `reminders` → `patients`
 - `treatment_prescriptions` → `treatments`
 - `notifications` → `users`
- **ON DELETE RESTRICT** : Pour les relations staff (empêche la suppression si des données liées existent)
 - `appointments` → `staff` (`dentist_id`)
 - `treatments` → `staff` (`dentist_id`)
- **ON DELETE SET NULL** : Pour certaines relations optionnelles
 - `users` → `patients` (`patient_id`)
 - `users` → `staff` (`staff_id`)
 - `admin_notifications` → `patients` (`patient_id`)

I.3.2. Contraintes d'Intégrité

- **Unicité** : Les emails doivent être uniques dans les tables `users`, `patients` et `staff`

- **Non-nullité** : Les champs critiques sont marqués NOT NULL pour garantir l'intégrité des données
- **ENUM** : Les champs de type ENUM garantissent que seules des valeurs prédéfinies peuvent être insérées
- **UNIQUE KEY** : La combinaison (`staff_id`, `day_of_week`) dans `staff_schedules` est unique

I.4. Vues

Le schéma inclut trois vues pour simplifier les requêtes fréquentes :

I.4.1. Vue `v_appointments_detail`

Cette vue combine les informations des rendez-vous avec les données des patients et des dentistes.

Colonnes :

- Toutes les colonnes de `appointments`
- `patient_id`, `patient_name`, `patient_phone`, `patient_email`
- `dentist_id`, `dentist_name`, `dentist_specialty`

Utilisation : Facilite l'affichage des rendez-vous avec toutes les informations nécessaires sans faire de jointures manuelles.

I.4.2. Vue `v_treatments_detail`

Cette vue combine les informations des traitements avec les données des patients et des dentistes.

Colonnes :

- Toutes les colonnes de `treatments`
- `patient_id`, `patient_name`
- `dentist_id`, `dentist_name`

Utilisation : Facilite l'affichage de l'historique des traitements avec les noms des patients et dentistes.

I.4.3. Vue `v_dashboard_stats`

Cette vue calcule les statistiques pour le tableau de bord.

Colonnes calculées :

- `total_patients` : Nombre total de patients
- `today_appointments` : Nombre de rendez-vous du jour
- `week_revenue` : Revenus de la semaine (somme des coûts des traitements des 7 derniers jours)
- `completion_rate` : Taux de complétion des rendez-vous sur les 30 derniers jours (pourcentage)

Utilisation : Fournit directement les statistiques pour le tableau de bord sans calculs complexes.

I.5. Optimisations

I.5.1. Index

Des index ont été créés sur les colonnes fréquemment utilisées dans les requêtes :

- **Index simples** : Sur les colonnes utilisées dans les clauses WHERE, JOIN, ORDER BY
- **Index composites** : Sur les combinaisons de colonnes fréquemment utilisées ensemble
- **Index uniques** : Sur les colonnes qui doivent être uniques (emails)

I.5.2. Types de Données

- **VARCHAR** : Pour les chaînes de caractères de longueur variable
- **TEXT** : Pour les textes longs (adresses, descriptions, notes)
- **DATE** : Pour les dates (format YYYY-MM-DD)
- **TIME** : Pour les heures (format HH :MM :SS)
- **DECIMAL(10,2)** : Pour les montants monétaires (précision à 2 décimales)
- **ENUM** : Pour les valeurs prédéfinies (rôles, statuts, types)
- **BOOLEAN** : Pour les valeurs booléennes (read/unread)

I.6. Données d'Exemple

Le schéma inclut des données d'exemple pour faciliter les tests et la démonstration :

- 5 patients avec leurs informations complètes
- 4 membres du personnel (2 dentistes, 1 assistant, 1 réceptionniste)
- Horaires de travail pour chaque membre du personnel
- 5 rendez-vous avec différents statuts

- 4 traitements avec leurs prescriptions
- 4 rappels en attente

Chapitre 5

Conclusion Générale

I. Résumé

Ce rapport a présenté l'analyse et la conception complète d'un système de gestion de cabinet dentaire. Les éléments principaux développés sont :

- **Analyse des besoins** : Identification des acteurs, fonctionnalités et contraintes du système
- **Modélisation UML** : Trois diagrammes essentiels (cas d'utilisation, séquence, classes) qui modélisent le système de manière complète
- **Conception de la base de données** : Schéma relationnel normalisé avec 11 tables, 3 vues et des contraintes d'intégrité robustes

II. Points Forts de la Conception

- **Modélisation complète** : Les diagrammes UML couvrent tous les aspects du système (fonctionnel, comportemental, structurel)
- **Base de données normalisée** : Le schéma respecte les principes de normalisation et évite la redondance
- **Intégrité des données** : Les contraintes de clés étrangères garantissent la cohérence des données
- **Performance** : Les index optimisent les requêtes fréquentes
- **Extensibilité** : L'architecture permet d'ajouter facilement de nouvelles fonctionnalités

III. Correspondance entre les Modèles

Il existe une correspondance claire entre les différents modèles :

- Les **acteurs** du diagramme de cas d'utilisation correspondent aux **classes User** et leurs spécialisations dans le diagramme de classes
- Les **cas d'utilisation** sont implémentés par les **méthodes** des classes
- Les **interactions** du diagramme de séquence correspondent aux **relations** entre classes
- Les **classes** du diagramme de classes correspondent aux **tables** de la base de données
- Les **attributs** des classes correspondent aux **colonnes** des tables
- Les **associations** entre classes correspondent aux **clés étrangères** dans la base de données

IV. Perspectives

Cette analyse et cette conception constituent une base solide pour l'implémentation du système. Les prochaines étapes pourraient inclure :

- Implémentation de l'interface utilisateur selon les cas d'utilisation identifiés
- Développement de l'API backend pour interagir avec la base de données
- Implémentation de la logique métier selon les diagrammes de séquence
- Tests de validation selon les scénarios modélisés
- Déploiement et mise en production